先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 17 – Deep Learning III: Training Practice and Applications

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

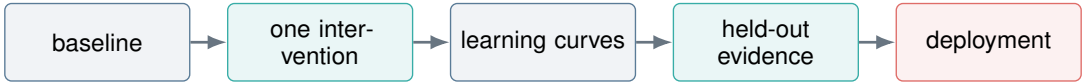
Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Treat training as a measurable dynamical system

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER



Keep a mechanism only when its gain survives seeds, splits, and system costs.

Guiding question

Which intervention changes optimization, generalization, or deployment value—and how will we know?

- optimizers shape update dynamics
- schedules define the budget
- normalization and regularization change signal flow
- data and deployment define usefulness

From SGD dynamics to a monitored application

By the end

You can reason about momentum and Adam, construct schedules, select normalization and regularization, audit augmentation, interpret learning curves, transfer representations, and define deployment evidence.

Period	Core content	Decision
1	optimizers and schedules	how should parameters move?
2	stabilization and data	what controls generalization?
3	transfer and deployment	does the model create target-domain value?

Record training settings and compare learning curves

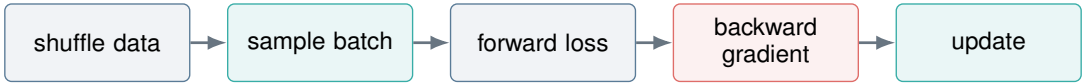
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

update dynamics | generalization evidence | operational value

What to calculate

Record the data version, split unit, optimizer state, sample budget, learning-rate path, gradient norms, train/validation curves, checkpoint rule, calibration, and system profile.

What does a mini-batch gradient estimate, and how does its noise affect progress?



The estimate is noisy but inexpensive; batch size controls variance, update frequency, memory, and hardware efficiency.

Question

What does a mini-batch gradient estimate, and how does its noise affect progress?

Example

training on video frames grouped by driving sequence

Main idea

The estimate is noisy but inexpensive; batch size controls variance, update frequency, memory, and hardware efficiency.

A mini-batch averages a few example gradients

- Full gradient: $g = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \ell_i$.
- Mini-batch gradient: $g_B = \frac{1}{|B|} \sum_{i \in B} \nabla_\theta \ell_i$.
- Example scalar gradients $2, -1, 3, -2$ average to 0.5.
- Uniform sampling gives an unbiased estimate; one batch can still differ greatly from the full gradient.



Mini-batch stochastic gradient descent: model and variables

Model and assumptions

Mini-batch SGD updates parameters using the average loss gradient over a sampled subset of training observations.

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_\theta \ell_i(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta_t g_t$$

Symbol	Meaning	Status
$\mathcal{B}_t$	mini-batch	sampled training cases
B	batch size	noise and memory control
η_t	learning rate	step scale

Mini-batch stochastic gradient descent: worked example

Quantity	Value	Meaning
per-sample gradients	$(2, -1, 3, 0)$	four cases
batch mean	1.0	SGD estimate
$\eta = 0.1$	-0.1	parameter change

What the result means

Choose batch construction from the sampling unit, then tune size for stable throughput and validation progress.

Mini-batch stochastic gradient descent: what to check

Main idea

The estimate is noisy but inexpensive; batch size controls variance, update frequency, memory, and hardware efficiency.

What to check

loss decrease per processed sample and gradient-noise scale

Limit

a batch built from correlated frames or repeated patients provides less independent information than its row count suggests

Mini-batch stochastic gradient descent: implementation

1. shuffle at the independent unit
2. form balanced batches if justified
3. average the loss
4. record gradient norm
5. compare samples per second

Check the result

Check loss decrease per processed sample and gradient-noise scale on training and held-out examples.

Common mistake

a batch built from correlated frames or repeated patients provides less independent information than its row count suggests

Mini-batch stochastic gradient descent: common mistakes

Example

training on video frames grouped by driving sequence

Explain the calculation and check loss decrease per processed sample and gradient-noise scale, using a small example.

Common mistake

a batch built from correlated frames or repeated patients provides less independent information than its row count suggests

What to do

Choose batch construction from the sampling unit, then tune size for stable throughput and validation progress.

Mini-batch stochastic gradient descent: check your understanding

Question

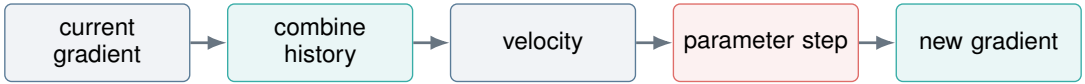
What does a mini-batch gradient estimate, and how does its noise affect progress?

Answer

The estimate is noisy but inexpensive; batch size controls variance, update frequency, memory, and hardware efficiency.

Decision. Choose batch construction from the sampling unit, then tune size for stable throughput and validation progress.

How does momentum accelerate consistent directions without amplifying every oscillation?



Consistent gradients accumulate along shallow valleys, while alternating components partially cancel.

Question

How does momentum accelerate consistent directions without amplifying every oscillation?

Example

optimizing a narrow curved loss valley

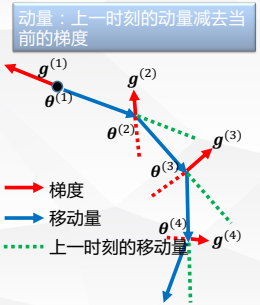
Main idea

Consistent gradients accumulate along shallow valleys, while alternating components partially cancel.

Momentum combines past and current gradients

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

移动方向：动量法



初始动量$\boldsymbol{v}^{(0)} = \mathbf{0}$

初始位置$\boldsymbol{\theta}^{(1)}$

计算$\boldsymbol{\theta}^{(1)}$处的梯度$\boldsymbol{g}^{(1)}$

动量$\boldsymbol{v}^{(1)} = \rho\boldsymbol{v}^{(0)} - \eta\boldsymbol{g}^{(1)}$

移到$\boldsymbol{\theta}^{(2)} = \boldsymbol{\theta}^{(1)} + \boldsymbol{v}^{(1)}$

计算$\boldsymbol{\theta}^{(2)}$处的梯度$\boldsymbol{g}^{(2)}$

动量$\boldsymbol{v}^{(2)} = \rho\boldsymbol{v}^{(1)} - \eta\boldsymbol{g}^{(2)}$

移到$\boldsymbol{\theta}^{(3)} = \boldsymbol{\theta}^{(2)} + \boldsymbol{v}^{(2)}$

移动量不仅与梯度有关，还与前一时刻的移动量有关。

Momentum and update geometry: model and variables

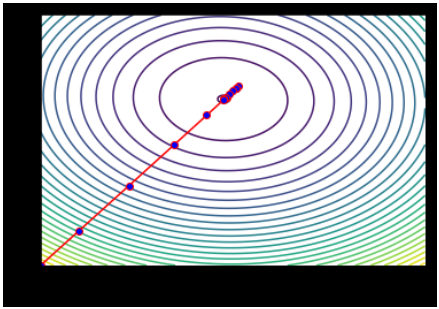
Model and assumptions

Momentum maintains a velocity that exponentially averages past gradients and uses it to update parameters.

$$v_{t+1} = \beta v_t + (1-\beta) g_t, \qquad \theta_{t+1} = \theta_t - \eta v_{t+1}$$

Symbol	Meaning	Status
v_t	velocity	smoothed direction
β	memory coefficient	effective history length
g_t	current gradient	new information

Momentum combines consistent updates and reduces oscillation



What is directly visible

Consistent gradients accumulate along shallow valleys, while alternating components partially cancel.

What the figure alone cannot prove

a learning rate stable for plain SGD can overshoot after momentum accumulates

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Momentum and update geometry: worked example

Quantity	Value	Meaning
previous velocity	1.0	same direction
new gradient	1.2	consistent
$\beta = 0.9$	$v = 1.02$	smoothed

What the result means

Tune momentum and learning rate as one dynamical system, not independent knobs.

The choice rule for Momentum and update geometry

Main idea

Consistent gradients accumulate along shallow valleys, while alternating components partially cancel.

What to check

validation progress per step and oscillation amplitude

Limit

a learning rate stable for plain SGD can overshoot after momentum accumulates

Momentum and update geometry: implementation

```
theta_m = np.array([4.0, -3.0])
velocity = np.zeros_like(theta_m)
for _ in range(220):
    grad = np.array([0.2*theta_m[0], 4.0*theta_m[1]])
    velocity = 0.9*velocity + 0.1*grad
    theta_m -= 0.12*velocity
print('Momentum quadratic solution:', theta_m)
```

Evidence and failure check

Check validation progress per step and oscillation amplitude with the fitted model and data split; reject the run if a learning rate stable for plain SGD can overshoot after momentum accumulates.

Momentum and update geometry: common mistakes

Example

optimizing a narrow curved loss valley

Explain the calculation and check validation progress per step and oscillation amplitude, using a small example.

Common mistake

a learning rate stable for plain SGD can overshoot after momentum accumulates

What to do

Tune momentum and learning rate as one dynamical system, not independent knobs.

Momentum and update geometry: check your understanding

Question

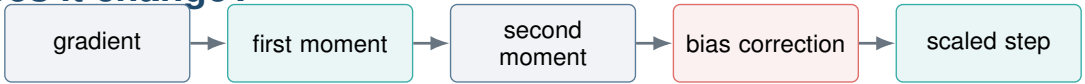
How does momentum accelerate consistent directions without amplifying every oscillation?

Answer

Consistent gradients accumulate along shallow valleys, while alternating components partially cancel.

Decision. Tune momentum and learning rate as one dynamical system, not independent knobs.

When should coordinate-wise gradient scaling help, and what does it change?



Adaptive scaling helps heterogeneous or sparse gradients, but changes implicit regularization and does not remove the need for a learning-rate schedule.

Question

When should coordinate-wise gradient scaling help, and what does it change?

Example

training an embedding model with sparse feature updates

Main idea

Adaptive scaling helps heterogeneous or sparse gradients, but changes implicit regularization and does not remove the need for a learning-rate schedule.

Adam keeps two moving averages and corrects their initial bias

- $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$.
- $\widehat{m}_t = m_t/(1-\beta_1^t)$, $\widehat{v}_t = v_t/(1-\beta_2^t)$.
- $\theta_{t+1} = \theta_t - \eta \widehat{m}_t/(\sqrt{\widehat{v}_t} + \epsilon)$.
- All products, squares, and divisions here are coordinate-wise.

Adaptive optimizers and Adam: model and variables

Model and assumptions

Adam combines momentum-like first moments with second-moment scaling and bias correction for each parameter coordinate.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \Delta\theta_t = -\eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

Symbol	Meaning	Status
m_t	first moment	direction estimate
v_t	second moment	coordinate scale
ϵ	numerical stabilizer	prevents division by zero

Adaptive optimizers and Adam: worked example

Quantity	Value	Meaning
gradient	$(0.01, 10)$	different scales
second moment	coordinatewise	normalizes magnitude
effective steps	closer scales	adaptive behavior

What the result means

Compare optimizers at matched compute and tuned schedules; prefer AdamW when decoupled weight decay is intended.

Adaptive optimizers and Adam: checking the result

Main idea

Adaptive scaling helps heterogeneous or sparse gradients, but changes implicit regularization and does not remove the need for a learning-rate schedule.

What to check

time-to-target and held-out performance

Limit

fast early optimization can be mistaken for better final generalization, especially with coupled L2 regularization

What to record when running Adaptive optimizers and Adam

```
theta_a = np.array([4.0, -3.0]); m = np.zeros(2); v = np.zeros(2)
for t in range(1, 101):
    g = np.array([0.2*theta_a[0], 4.0*theta_a[1]])
    m = 0.9*m + 0.1*g; v = 0.999*v + 0.001*g*g
    mh = m/(1-0.9**t); vh = v/(1-0.999**t)
    theta_a -= 0.08*mh/(np.sqrt(vh)+1e-8)
print('Adam quadratic solution:', theta_a)
```

Evidence and failure check

Check time-to-target and held-out performance with the fitted model and data split; reject the run if fast early optimization can be mistaken for better final generalization, especially with coupled L2 regularization.

The warning sign for Adaptive optimizers and Adam

Example

training an embedding model with sparse feature updates

Explain the calculation and check time-to-target and held-out performance, using a small example.

Common mistake

fast early optimization can be mistaken for better final generalization, especially with coupled L2 regularization

What to do

Compare optimizers at matched compute and tuned schedules; prefer AdamW when decoupled weight decay is intended.

Explain Adaptive optimizers and Adam without naming the algorithm

Question

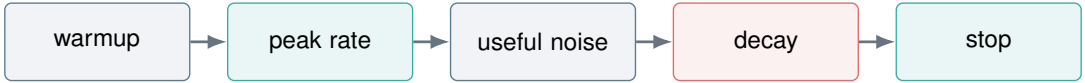
When should coordinate-wise gradient scaling help, and what does it change?

Answer

Adaptive scaling helps heterogeneous or sparse gradients, but changes implicit regularization and does not remove the need for a learning-rate schedule.

Decision. Compare optimizers at matched compute and tuned schedules; prefer AdamW when decoupled weight decay is intended.

How should step size change as the model moves from exploration to refinement?



Warmup protects unstable early statistics; decay reduces late noise; schedule comparisons require matched step or sample budgets.

Question

How should step size change as the model moves from exploration to refinement?

Example

fine-tuning a pretrained model with a new randomly initialized head

Main idea

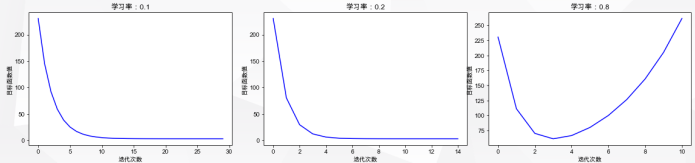
Warmup protects unstable early statistics; decay reduces late noise; schedule comparisons require matched step or sample budgets.

The learning rate sets the update size

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

学习率

- 学习率对训练的影响
  - 目标函数变化太慢：学习率太低
  - 目标函数出现NaN：通常意味着学习率太高
  - 建议：[1e-3 ... 1e-5]



How Learning-rate schedules, warmup, and stopping changes the representation

Model and assumptions

A learning-rate schedule defines the optimization horizon through warmup, a high-learning phase, decay, and a stopping rule.

$$\eta_t = \eta_{min} + \tfrac{1}{2}(\eta_{max} - \eta_{min}) \left[1 + \cos\left(\pi \frac{t - t_w}{T - t_w}\right)\right]$$

Symbol	Meaning	Status
t_w	warmup length	early stabilization
T	training horizon	budget
η_{min}, η_{max}	schedule bounds	late and peak steps

A learning-rate schedule changes step size over training

- Update: $\theta_{t+1} = \theta_t - \eta_t g_t$.
- Warmup increases η_t gradually at the beginning.
- Decay reduces η_t later, for example using a cosine or step schedule.
- Compare loss curves and actual update sizes; the schedule alone does not guarantee convergence.



Check the direction of change in Learning-rate schedules, warmup, and stopping

Quantity	Value	Meaning
steps	$T = 1000$	budget
warmup	$t_w = 50$	five percent
final rate	$0.01\eta_{max}$	refinement

What the result means

Define the schedule on an explicit update or sample budget and report both.

When Learning-rate schedules, warmup, and stopping earns a place in the pipeline

Main idea

Warmup protects unstable early statistics; decay reduces late noise; schedule comparisons require matched step or sample budgets.

What to check

validation loss versus processed samples

Limit

comparing schedules by epoch when batch sizes differ compares different numbers of updates and samples

Learning-rate schedules, warmup, and stopping: implementation

```
def cosine_lr(step, total=100, warmup=10, peak=0.1, floor=0.001):
    if step < warmup: return peak*(step+1)/warmup
    phase = (step-warmup)/(total-warmup)
    return floor + 0.5*(peak-floor)*(1+np.cos(np.pi*phase))
rates = np.array([cosine_lr(t) for t in range(100)])
print('Schedule start/peak/end:', rates[0], rates.max(), rates[-1])
```

Evidence and failure check

Check validation loss versus processed samples with the fitted model and data split; reject the run if comparing schedules by epoch when batch sizes differ compares different numbers of updates and samples.

When the objective of Learning-rate schedules, warmup, and stopping is not enough

Example

fine-tuning a pretrained model with a new randomly initialized head

Explain the calculation and check validation loss versus processed samples, using a small example.

Common mistake

comparing schedules by epoch when batch sizes differ compares different numbers of updates and samples

What to do

Define the schedule on an explicit update or sample budget and report both.

A final decision on Learning-rate schedules, warmup, and stopping

Question

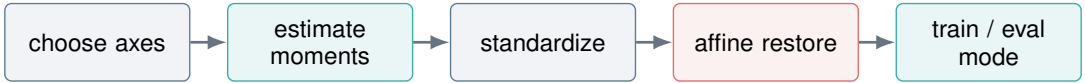
How should step size change as the model moves from exploration to refinement?

Answer

Warmup protects unstable early statistics; decay reduces late noise; schedule comparisons require matched step or sample budgets.

Decision. Define the schedule on an explicit update or sample budget and report both.

Which axes should be normalized, and which statistics must remain stable at inference?



The normalization axes determine which variation is removed, and train-eval behavior is part of the model state.

Question

Which axes should be normalized, and which statistics must remain stable at inference?

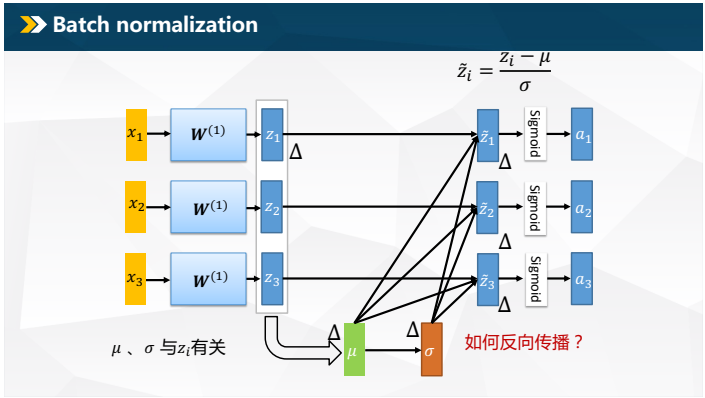
Example

serving a CNN one image at a time

Main idea

The normalization axes determine which variation is removed, and train-eval behavior is part of the model state.

Batch normalization uses batch statistics



The variables that make Batch normalization and layer normalization work

Model and assumptions

Batch normalization uses training-batch feature statistics and running estimates; layer normalization normalizes features within each example.

$$\widehat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \qquad y = \gamma \widehat{x} + \beta$$

Symbol	Meaning	Status
μ, σ^2	selected-axis statistics	batch or example dependent
γ, β	learned affine terms	restore useful scale and shift
running stats	inference state	BatchNorm only

Read the numbers produced by Batch normalization and layer normalization

Quantity	Value	Meaning
BatchNorm	per channel over batch + space	CNN convention
LayerNorm	per example over features	sequence convention
batch size 1	unstable BN estimate	prefer alternatives

What the result means

Choose normalization from tensor semantics and deployment batch behavior, then test mode switching explicitly.

Three conditions for trusting Batch normalization and layer normalization

Main idea

The normalization axes determine which variation is removed, and train-eval behavior is part of the model state.

What to check

activation statistics and train-eval prediction drift

Limit

forgetting evaluation mode makes BatchNorm depend on served batch composition and can change one sample's prediction

An implementation checklist for Batch normalization and layer normalization

1. write the normalized axes
2. check train and evaluation modes
3. monitor running statistics
4. test deployment batch size
5. compare GroupNorm or LayerNorm when needed

Check the result

Check activation statistics and train-eval prediction drift on training and held-out examples.

Common mistake

forgetting evaluation mode makes BatchNorm depend on served batch composition and can change one sample's prediction

Reject Batch normalization and layer normalization under this condition

Example

serving a CNN one image at a time

Explain the calculation and check activation statistics and train-eval prediction drift, using a small example.

Common mistake

forgetting evaluation mode makes BatchNorm depend on served batch composition and can change one sample's prediction

What to do

Choose normalization from tensor semantics and deployment batch behavior, then test mode switching explicitly.

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Batch normalization and layer normalization: check your understanding

Question

Which axes should be normalized, and which statistics must remain stable at inference?

Answer

The normalization axes determine which variation is removed, and train-eval behavior is part of the model state.

Decision. Choose normalization from tensor semantics and deployment batch behavior, then test mode switching explicitly.

Which regularizer targets parameter scale, co-adaptation, or training horizon?



These mechanisms act differently and must be evaluated with the optimizer, normalization, and data scale.

Question

Which regularizer targets parameter scale, co-adaptation, or training horizon?

Example

regularizing a medium-sized classifier on limited labeled data

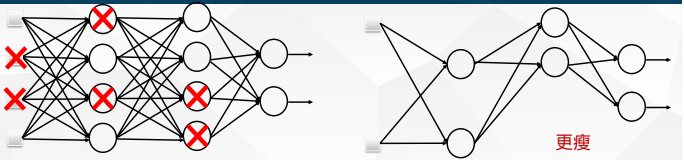
Main idea

These mechanisms act differently and must be evaluated with the optimizer, normalization, and data scale.

Dropout randomly removes training activations

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Dropout



在每个Mini- Batch训练时

- 随机删除一部分神经元，即随机地将输入中的每个元素以概率p置0

$$r_j^{(l)} = Bernoulli(1-p)$$

使得$\mathbb{E}[a] = \mathbb{E}[\text{dropout}(a)]$：对下一层网络而言，输入的幅值“看起来”一样

- 前向传播

$$\tilde{\boldsymbol{a}}^{(l)} = \boldsymbol{r}^{(l)} \odot \boldsymbol{a}^{(l)}, z_i^{(l+1)} = \boldsymbol{w}_i^{(l)} \tilde{\boldsymbol{a}}^{(l)} + b_i^{(l+1)}, a_i^{(l+1)} = \left(\frac{1}{1-p}\right) \sigma\left(z_i^{(l+1)}\right)$$

- 反向传播损失，更新参数（没有被删除的那一部分神经元的参数得到更新，删除的神经元参数保持被删除前的结果）

Weight decay, dropout, and capacity control: model and variables

Model and assumptions

Weight decay shrinks parameters during updates, dropout samples subnetworks during training, and early stopping limits the optimization path.

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\, g_t, \qquad \widetilde{a} = \frac{m \odot a}{1 - p}$$

Symbol	Meaning	Status
λ	decoupled decay	parameter shrinkage
$m \sim \mathrm{Bernoulli}(1-p)$	dropout mask	sampled subnetwork
p	drop probability	noise strength

Weight decay, dropout, and capacity control: worked example

Quantity	Value	Meaning
weight decay	10^{-4}	small continuous shrink
dropout	$p = 0.2$	stochastic units
evaluation	dropout off	expected scaling preserved

What the result means

Add the minimum regularization that improves repeated validation, one mechanism at a time.

A decision card for Weight decay, dropout, and capacity control

Main idea

These mechanisms act differently and must be evaluated with the optimizer, normalization, and data scale.

What to check

validation loss and training-validation gap

Limit

strong dropout plus BatchNorm can distort activation statistics, while excessive decay underfits small heads

Reproduce the result of Weight decay, dropout, and capacity control

1. establish an unregularized baseline
2. add decoupled weight decay
3. use dropout only where justified
4. track train-validation gap
5. verify inference-mode behavior

Check the result

Check validation loss and training-validation gap on training and held-out examples.

Common mistake

strong dropout plus BatchNorm can distort activation statistics, while excessive decay underfits small heads

A stress test for Weight decay, dropout, and capacity control

Example

regularizing a medium-sized classifier on limited labeled data

Explain the calculation and check validation loss and training-validation gap, using a small example.

Common mistake

strong dropout plus BatchNorm can distort activation statistics, while excessive decay underfits small heads

What to do

Add the minimum regularization that improves repeated validation, one mechanism at a time.

Weight decay, dropout, and capacity control: check your understanding

Question

Which regularizer targets parameter scale, co-adaptation, or training horizon?

Answer

These mechanisms act differently and must be evaluated with the optimizer, normalization, and data scale.

Decision. Add the minimum regularization that improves repeated validation, one mechanism at a time.

Which transformations preserve the label and deployment meaning?



Every augmentation is a domain claim; transforms must also update boxes, masks, landmarks, or sequence labels consistently.

Question

Which transformations preserve the label and deployment meaning?

Example

augmenting handwritten symbols and medical images

Main idea

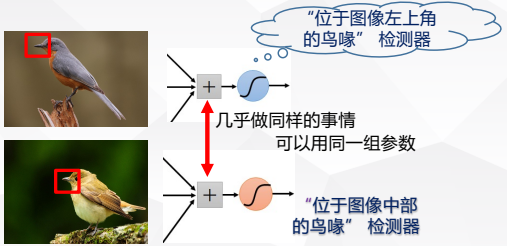
Every augmentation is a domain claim; transforms must also update boxes, masks, landmarks, or sequence labels consistently.

Translation can preserve an image label

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

对图像，为什么要用CNN？

- 同一个模式会在图像中不同的区域出现。



Data augmentation and label preservation: model and variables

Model and assumptions

Data augmentation samples label-preserving transformations to expand observed variation and encode expected invariances.

$$\min_{\theta} \ \mathbb{E}_{(x,y)}\mathbb{E}_{t\sim\mathcal{T}}\left[L(y_t, f_\theta(t(x)))\right]$$

Symbol	Meaning	Status
$\mathcal{T}$	transformation distribution	assumed invariances
$t(x)$	transformed input	synthetic observation
y_t	transformed target	task-consistent label

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Data augmentation and label preservation: worked example

Quantity	Value	Meaning
handwritten digit	small rotation	usually valid
digit 6	180-degree rotation	may become 9
segmentation crop	crop mask too	aligned target

What the result means

Use only transformations whose label semantics are defensible and verified visually.

Data augmentation and label preservation: checking the result

Main idea

Every augmentation is a domain claim; transforms must also update boxes, masks, landmarks, or sequence labels consistently.

What to check

clean and transformed validation performance

Limit

an apparently harmless flip or crop can change laterality, orientation, or remove the labeled object

Data augmentation and label preservation: implementation

1. list deployment variations
2. classify invariant versus label-changing transforms
3. visualize transformed pairs
4. set realistic magnitudes
5. ablate and stress-test

Check the result

Check clean and transformed validation performance on training and held-out examples.

Common mistake

an apparently harmless flip or crop can change laterality, orientation, or remove the labeled object

What breaks first in Data augmentation and label preservation

Example

augmenting handwritten symbols and medical images

Explain the calculation and check clean and transformed validation performance, using a small example.

Common mistake

an apparently harmless flip or crop can change laterality, orientation, or remove the labeled object

What to do

Use only transformations whose label semantics are defensible and verified visually.

Data augmentation and label preservation: check your understanding

Question

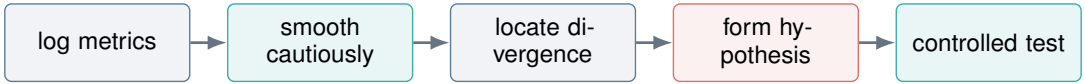
Which transformations preserve the label and deployment meaning?

Answer

Every augmentation is a domain claim; transforms must also update boxes, masks, landmarks, or sequence labels consistently.

Decision. Use only transformations whose label semantics are defensible and verified visually.

What can training and validation trajectories tell us about optimization, capacity, and data?



The shape and timing of divergence are more diagnostic than one final checkpoint.

Question

What can training and validation trajectories tell us about optimization, capacity, and data?

Example

a model whose validation loss rises while accuracy stays flat

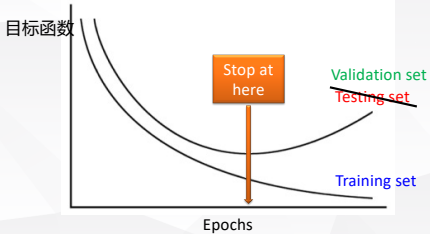
Main idea

The shape and timing of divergence are more diagnostic than one final checkpoint.

Validation loss tells us when to stop

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Early Stopping



先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Learning curves and failure localization: model and variables

Model and assumptions

Learning curves plot losses and task metrics against updates or samples to localize underfitting, overfitting, instability, and distribution mismatch.

$$G_t = L_{valid}(\theta_t) - L_{train}(\theta_t), \qquad t^* = \arg\min_t L_{valid}(\theta_t)$$

Symbol	Meaning	Status
G_t	generalization gap	time-dependent
t^*	best checkpoint	validation selected
x-axis	updates or samples	comparison budget

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Learning curves and failure localization: worked example

Quantity	Value	Meaning
both losses high	underfit or optimization	increase fit ability
train down; valid up	overfit or shift	regularize or audit
spikes	unstable updates	lower rate or inspect batches



What the result means

Keep raw traces and use curves to choose the next falsifiable diagnostic, not to tell a post-hoc story.

Accept or reject Learning curves and failure localization

Main idea

The shape and timing of divergence are more diagnostic than one final checkpoint.

What to check

best validation point, gap, and run-to-run variance

Limit

heavy smoothing can hide spikes and make an unstable run appear monotonic

Learning curves and failure localization: implementation

```
from sklearn.neural_network import MLPClassifier
from sklearn.exceptions import ConvergenceWarning
import warnings
model = MLPClassifier(hidden_layer_sizes=(48,), activation='relu', solver='sgd', learning_rate_init=0.08, momentum
     =0.9, alpha=1e-4, max_iter=1, warm_start=True, random_state=7)
curve = []
with warnings.catch_warnings():
    warnings.simplefilter('ignore', ConvergenceWarning)
    for epoch in range(35):
        model.fit(X_train, y_train)
        curve.append((model.loss_, model.score(X_valid, y_valid)))
print('MLP first/last:', tuple(round(v,3) for v in curve[0]), tuple(round(v,3) for v in curve[-1]))
```

Evidence and failure check

Check best validation point, gap, and run-to-run variance with the fitted model and data split; reject the run if heavy smoothing can hide spikes and make an unstable run appear monotonic.

The hidden assumption in Learning curves and failure localization

Example

a model whose validation loss rises while accuracy stays flat

Explain the calculation and check best validation point, gap, and run-to-run variance, using a small example.

Common mistake

heavy smoothing can hide spikes and make an unstable run appear monotonic

What to do

Keep raw traces and use curves to choose the next falsifiable diagnostic, not to tell a post-hoc story.

Learning curves and failure localization: check your understanding

Question

What can training and validation trajectories tell us about optimization, capacity, and data?

Answer

The shape and timing of divergence are more diagnostic than one final checkpoint.

Decision. Keep raw traces and use curves to choose the next falsifiable diagnostic, not to tell a post-hoc story.

Which pretrained features should be frozen, adapted, or replaced for a new domain?



Question

Which pretrained features should be frozen, adapted, or replaced for a new domain?

Example

adapting a natural-image backbone to a specialist imaging domain

Main idea

Early features may transfer broadly, later features and normalization statistics are task-specific, and negative transfer is measurable.

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Pretraining supplies features for a new task

预训练

- 预训练（pre-training）是一种非常有效的神经网络的初始化方法。
- 非监督：贪心地逐层训练自编码器得到初始权重，然后再做细调（不再常用）
- 类似问题中已经训练好的模型
  - 若数据集与预训练模型采用的训练数据集非常相似，且新数据集较小，只要在预训练模型最顶层输出特征上再训练一个线性分类器即可
  - 若新数据集较大，可以使用一个较小的学习速率对预训练模型的最后几个顶层进行调优。
  - 如果新数据集与预训练模型采用的训练数据集相差较大但拥有足够多的数据，则需要对网络的多个层进行调优，同样也要使用较小的学习速率。
  - 最坏的情况是新数据集不仅较小，而且与原始数据集相差较大，此时基于较为靠前的特征层就使用SVM分类器可能是相对较好的方案 。

Transfer learning and representation reuse: model and variables

Model and assumptions

Transfer learning initializes a target model from parameters learned on a source task, then adapts selected layers using target data.

$$\theta^* = \arg\min_{\theta} L_{target}(\theta) + \lambda \|\theta - \theta_{source}\|_2^2$$

Symbol	Meaning	Status
θ_{source}	pretrained parameters	source representation
L_{target}	target loss	new evidence
λ	stay-close strength	adaptation control

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL ENGINEERING RESEARCH CENTER

Transfer learning and representation reuse: worked example

Quantity	Value	Meaning
target labels	500	limited data
frozen backbone	linear probe	representation test
unfreeze top block	small learning rate	controlled adaptation

What the result means

Retain pretraining only when repeated target validation beats both scratch and frozen baselines.

Separate a good fit from a good decision in Transfer learning and representation reuse

Main idea

Early features may transfer broadly, later features and normalization statistics are task-specific, and negative transfer is measurable.

What to check

gain over training from scratch and over a frozen probe

Limit

source shortcuts or incompatible normalization statistics can create negative transfer despite faster convergence

Transfer learning and representation reuse: implementation

1. match input semantics
2. evaluate a frozen linear probe
3. replace the task head
4. unfreeze from top to bottom
5. use discriminative learning rates

Check the result

Check gain over training from scratch and over a frozen probe on training and held-out examples.

Common mistake

source shortcuts or incompatible normalization statistics can create negative transfer despite faster convergence

Do not trust Transfer learning and representation reuse before this check

Example

adapting a natural-image backbone to a specialist imaging domain

Explain the calculation and check gain over training from scratch and over a frozen probe, using a small example.

Common mistake

source shortcuts or incompatible normalization statistics can create negative transfer despite faster convergence

What to do

Retain pretraining only when repeated target validation beats both scratch and frozen baselines.

Transfer learning and representation reuse: check your understanding

Question

Which pretrained features should be frozen, adapted, or replaced for a new domain?

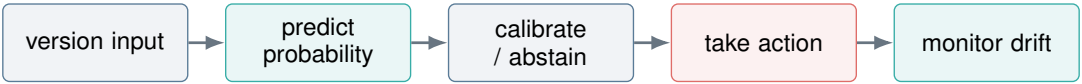
Answer

Early features may transfer broadly, later features and normalization statistics are task-specific, and negative transfer is measurable.

Decision. Retain pretraining only when repeated target validation beats both scratch and frozen baselines.

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Check accuracy, calibration, and cost before deployment



An application succeeds only when the complete pipeline remains useful under the served population and operating conditions.

Question

What evidence is required to turn a trained network into a responsible operational component?

Example

using a deep model to prioritize time-sensitive expert review

Main idea

An application succeeds only when the complete pipeline remains useful under the served population and operating conditions.

Check training and inference behavior explicitly

- Dropout samples masks during training and is disabled during inference.
- Batch normalization usually uses batch statistics in training and running statistics in inference.
- Layer normalization uses each example's feature statistics in both modes.
- Evaluate the saved model in inference mode on held-out examples.



Applications, calibration, and deployment monitoring: model and variables

Model and assumptions

Deployment wraps a model with preprocessing, decision thresholds, uncertainty handling, latency constraints, monitoring, and a rollback policy.

$$\text{NetBenefit}(\tau) = \text{TP}(\tau) V_{TP} - \text{FP}(\tau) C_{FP} - C_{system}$$

Symbol	Meaning	Status
τ	decision threshold	operating point
V_{TP}, C_{FP}	outcome value and cost	domain inputs
C_{system}	latency and operational cost	end-to-end burden

Applications, calibration, and deployment monitoring: worked example

Quantity	Value	Meaning
model score	0.72	raw output
calibrated risk	0.43	decision input
threshold	0.35	trigger review

What the result means

Deploy with input and outcome monitoring, human escalation, and a tested rollback path.

Applications, calibration, and deployment monitoring: checking the result

Main idea

An application succeeds only when the complete pipeline remains useful under the served population and operating conditions.

What to check

net benefit, calibration, subgroup recall, and tail latency

Limit

silent preprocessing drift can preserve uptime while systematically shifting probabilities and actions

Applications, calibration, and deployment monitoring: implementation

1. freeze a versioned pipeline
2. calibrate on representative held-out data
3. select threshold from utility
4. load-test tail latency
5. monitor drift and delayed outcomes

Check the result

Check net benefit, calibration, subgroup recall, and tail latency on training and held-out examples.

Common mistake

silent preprocessing drift can preserve uptime while systematically shifting probabilities and actions

Applications, calibration, and deployment monitoring: limitations

Example

using a deep model to prioritize time-sensitive expert review

Explain the calculation and check net benefit, calibration, subgroup recall, and tail latency, using a small example.

Common mistake

silent preprocessing drift can preserve uptime while systematically shifting probabilities and actions

What to do

Deploy with input and outcome monitoring, human escalation, and a tested rollback path.

Applications, calibration, and deployment monitoring: check your understanding

Question

What evidence is required to turn a trained network into a responsible operational component?

Answer

An application succeeds only when the complete pipeline remains useful under the served population and operating conditions.

Decision. Deploy with input and outcome monitoring, human escalation, and a tested rollback path.

Diagnose the curve before changing the optimizer

Decision

Separate underfitting, overfitting, unstable updates, data leakage, and metric mismatch from the shapes of training and validation traces.

Evidence

Change one mechanism at a time and keep the original run as a control.

Stop condition

Do not stack several tricks after one bad epoch; the result will not identify the cause.

Transfer, calibrate, then monitor

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Decision

Freeze and unfreeze by evidence, recalibrate probabilities on the target domain, and define drift slices before launch.

Evidence

Record accuracy, calibration, latency, and abstention together.

Stop condition

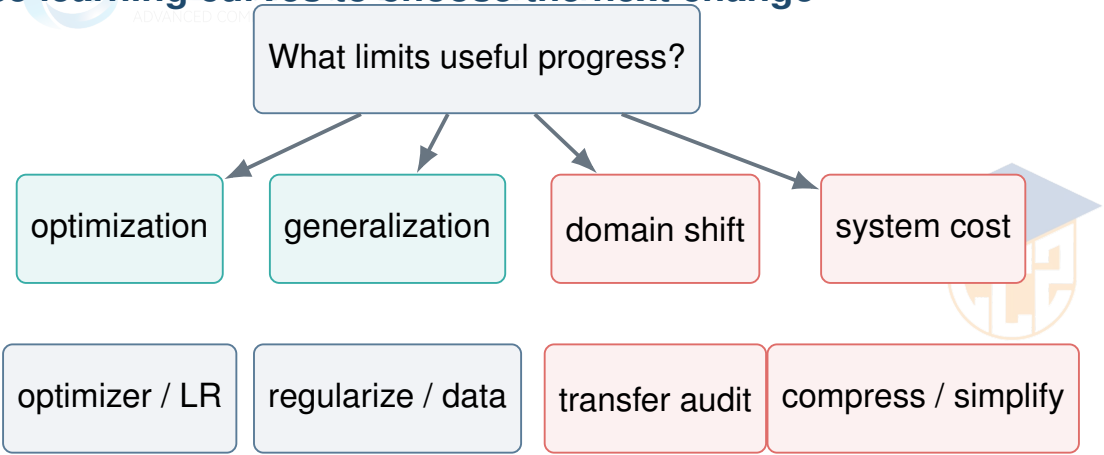
Roll back when a high-confidence error slice grows even if the aggregate score is unchanged.

Compare training mechanisms by what they modify

Mechanism	Primary effect	First diagnostic
momentum / Adam	update geometry	progress and oscillation
schedule	step-size horizon	loss versus samples
normalization / decay	signal and capacity	statistics and gap
augmentation / transfer	effective data and prior	controlled target validation



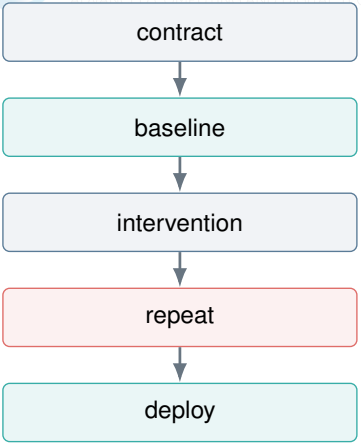
Use learning curves to choose the next change



Start from the deliverable

Localize whether the present limit is unstable updates, underfit capacity, overfit gap, domain mismatch, or deployment cost.

A reliable deep-training workflow



Optimization evidence and deployment evidence answer different questions.

1. freeze data and split contracts
2. establish a deterministic tiny-batch test
3. run a documented baseline
4. change one mechanism at a time
5. repeat seeds and target-domain validation
6. calibrate, load-test, and define rollback



Measure benefit at matched resource budgets

$$\Delta U = U_{new} - U_{base}, \qquad \mathrm{SE}(\Delta U) = s_\Delta / \sqrt{R}$$

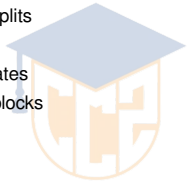
Observation	Likely issue	Next action
loss spikes	step too large or bad batch	inspect LR and data
large train-valid gap	capacity or shift	regularize and audit split
good metric; poor net value	threshold or system cost	optimize decision pipeline

Stress-test training and serving

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

training dynamics	target-domain evidence	operational resilience

- repeat seeds and dependency-aware splits
- compare by samples, time, and energy
- inspect activations, gradients, and updates
- ablate augmentations and transferred blocks
- audit calibration and subgroups
- simulate drift, overload, and rollback



先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Six takeaways to retain

1. mini-batches trade noise for throughput
2. momentum and learning rate are coupled
3. Adam adapts scale but still needs a schedule
4. normalization axes define behavior
5. augmentation is an invariance claim
6. deployment requires calibration and monitoring

Exit ticket: choose the next training intervention

Prompt

State the observed trace, one hypothesized mechanism, one controlled change, one validation metric, and the condition under which you would revert it.

Case

Training loss falls smoothly, validation loss bottoms out early, and a transferred model converges twice as fast but has worse calibration on a new hospital site.